DS5102: Big Data Lab

Lec 01

Nikhil Krishnan M

nikhil@iitpkd.ac.in
https://iitpkd.ac.in/people/nikhil

Evaluation Pattern

- Lab Experiments: 60%

- Final exam: 40%

Big Data?

- Characterized by multiple V's
  - Volume
  - Velocity
  - Variety
- Data so massive that it cannot be stored/processed by a single machine!

Big Data: Challenges

- How to store data?
- How to perform computations on data?
- How to scale?

Storing Big Data

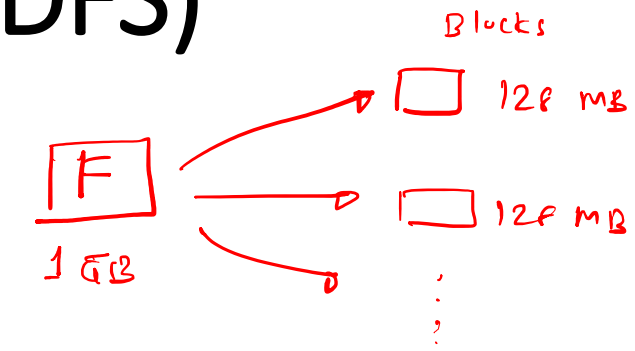
- Traditional databases struggle with big data.

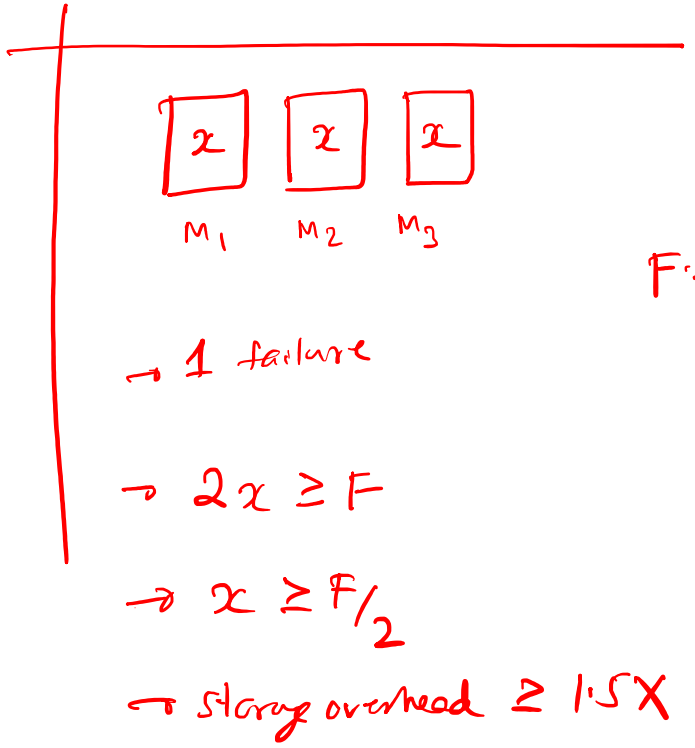
- Need for distributed storage systems

Hadoop Distributed File System (HDFS)

- A popular distributed file system
- Stores data in blocks (say, 128 MB each).
- Each block has replication 3 (by default).
  - Provides reliability!

Data locality





HDFS Architecture

- Master-Slave architecture
- Master → NameNode
  - Manages metadata
- Slaves → DataNodes
  - Deals with the storage of actual data

Computations on Big Data

- Traditional algorithms fail
- Two models:
  - Parallel model
  - Streaming model

Parallel Model: MapReduce

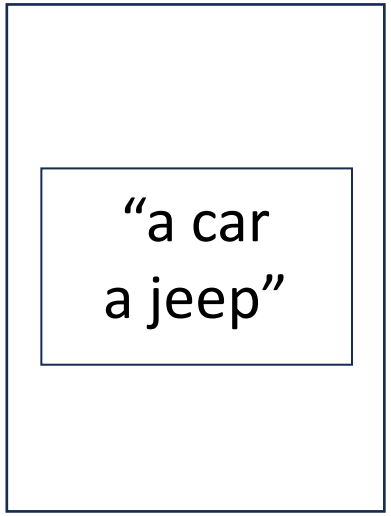
- Google (2004)

- Main computing model used in Hadoop

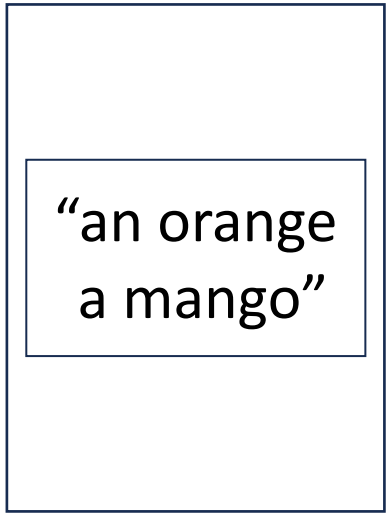
MapReduce: Toy Example

- Suppose you have a file **F** with contents:
    - “a car a jeep an orange a mango”
    - Block 1: “a car a jeep”
    - Block 2: “an orange a mango”

MapReduce (contd.)

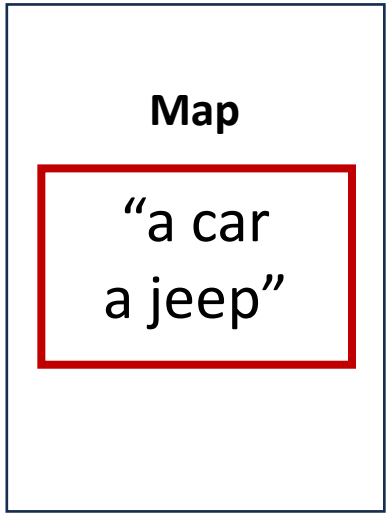


Worker 1

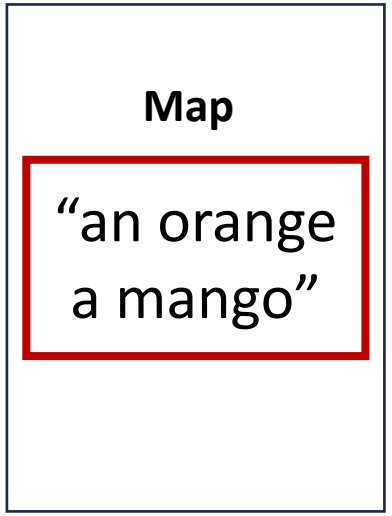


Worker 2

MapReduce (contd.)

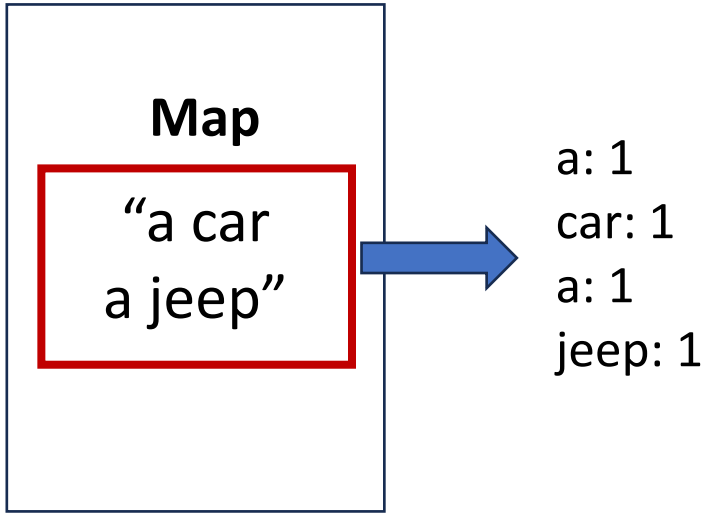


Worker 1

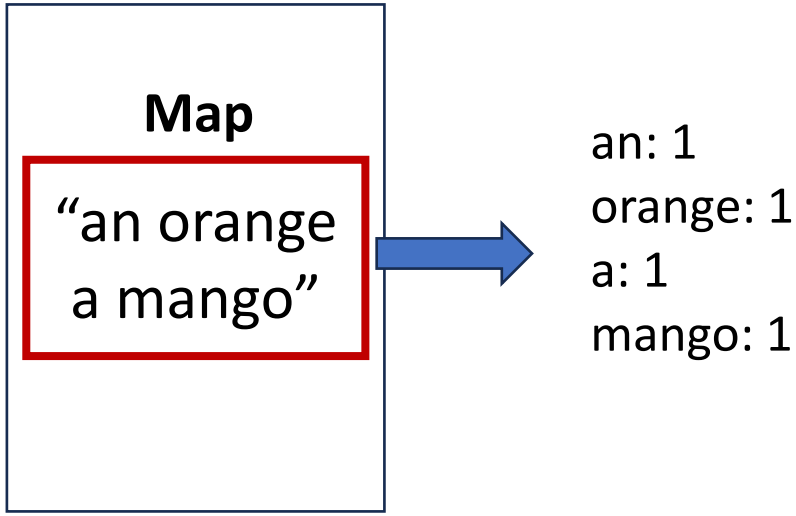


Worker 2

MapReduce (contd.)



Worker 1



Worker 2

will generate key-value pairs!

MapReduce (contd.)

a: 1
car: 1
a: 1
jeep: 1

a: 1
orange: 1
an: 1
mango: 1

Shuffle

a: 1
a: 1
a: 1

an: 1

car: 1

jeep: 1

orange: 1

mango: 1

MapReduce (contd.)

a: 1
car: 1
a: 1
jeep: 1

a: 1
orange: 1
an: 1
mango: 1

Shuffle

a: 1
a: 1
a: 1

an: 1

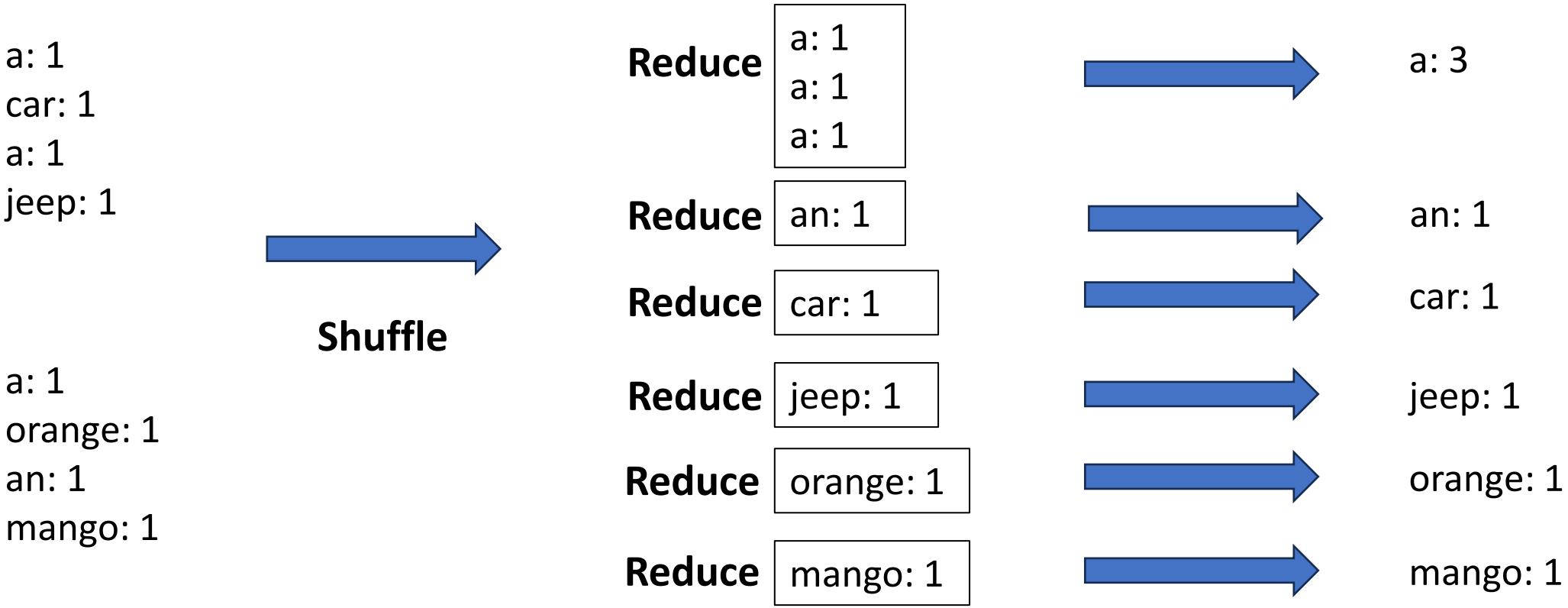
car: 1

jeep: 1

orange: 1

mango: 1

MapReduce (contd.)



Streaming Model

- Data processed continuously as received.

- Limited storage (compared to the size of data)

Streaming Algorithm: Example

Frequent elements algo.

n: unknown

Goal: You have a "stream" of n integers $\{x_i\}_{i=1}^{n}$

very large

You need to identify all those integers which appear $> \left(\frac{n}{k}\right)$ times $\quad k \in \{2, 3, \ldots\}$

Constraint: You can store only k integers & corresp. counters

O/p: A set $\mathcal{H}$ which contains all the integers appearing $> (n/k)$ times

[It is okay even if there are some integers with $\leq (n/k)$ frequency also feature in the list!]

Frequent elements algo. [Misra-Gries]

$\mathcal{H} = \phi$ $\qquad$ $C[h] = 0 \;\; \forall \; h \notin \mathcal{H}$

$\{x_i\}_{i=1}^{n}$

For each x_i:

if $x_i \notin \mathcal{H}$:

Add x_i to $\mathcal{H}$

$C[x_i] = 1$

else:

$C[x_i]$++

if $|\mathcal{H}| = k$:

Decrement all non-zero counters by 1

Remove from $\mathcal{H}$ all h's which have $C[h] = 0$

(*)

Return $\mathcal{H}$

$k = 4$

$\boxed{1, 2, 3, 4, 5, 6, 7, 1}, 1, 1, 1, \ldots$

①	②	...	④	...	⑦	⑧
$H = \{1\}$	$H = \{1, 2\}$	...	$H = \{1, 2, 3, 4\}$		$H = \{5, 6, 7\}$	Reset
$C[1] = 1$	$C[1] = 1$		$C[1] = 1$		$C[5] = C[6] = C[7] = 1$	
	$C[2] = 1$		$C[2] = 1$			
			$C[3] = 1$			
			$C[4] = 1$			
			$\downarrow$ "Pruning"			
			$H = \phi$			

⑨

- All the counters cumulatively increment precisely n times

- In segment ⊛, precisely k decrements happen

- Total # decrements $\leq$ total # increments $= n$

- Thus, the segment ⊛ can run at most (n/k) times

$\Rightarrow$ Each counter can decrement at most (n/k) times

- For any element x which has occurred $> (n/k)$ times, $C[x]$ has incremented $> (n/k)$ times

- Consider a "social network" graph. Each node x_i denotes a person
- $(x_i, x_j) \rightarrow x_i$ is friends with x_j
- Design map-reduce fns. to count the #friends of each person

Thank You!